Robot Dynamics Midterm 2

Prof. Marco Hutter
Teaching Assistants: Kaixian Qu, Changan Chen, Luca Monegaglia, Chenlu Shou, Filip Bjelonic, Francesca Bray, Adria Lopez

November 5, 2025

Duration: 1h 15min
Permitted Aids: The exam is open book, which means you can use the script, slides, exercises, etc; The use of internet (besides for licenses) is forbidden; no communication among students during the test is allowed.

1 Instructions

1. Download the ZIP file `RobotDynamics_Quiz2_2025.zip` from Moodle. Extract all contents of this file into a new folder and set MATLAB's[1] current path to this folder.
2. Run `init_workspace` in the MATLAB command line.
3. All problem files that you need to complete are located in the `problems` folder.
4. Run `evaluate_problems` to check if your functions run. This script does not test for correctness. You will get 0 points if a function does not run (e.g., for syntax errors) or does not terminate (e.g. infinite loops). Remember to rerun the setup section before testing new functions.
5. When the time is up, zip the entire folder and name it `ETHStudentID_StudentName.zip`, for example, `01234567_JohnDoe.zip`. Submit this zip-file through Moodle under **Midterm Exam 2 Submission**. You should receive a confirmation email.
6. If the previous step did not succeed, you can email your file to `robotdynamics@leggedrobotics.com` from your ETH email address with the subject line `[RobotDynamics] ETHStudentID - StudentName`
7. **Important:**
   (a) Implementations outside the provided templates will not be graded and receive 0 points. No external libraries are allowed, except for MATLAB's Symbolic Math Toolbox.
   (b) Helper functions included in the `solutions/pcode` directory are used for simulating the robot. Using these functions to solve the questions (unless explicitly stated) is prohibited and will lead to 0 points.
   (c) Helper functions in the `utils` folder should be used by students in their solutions when needed. The function body can be freely inspected, however modifications are prohibited and will lead to 0 points.

[1] Online version of MATLAB at `https://matlab.mathworks.com/`

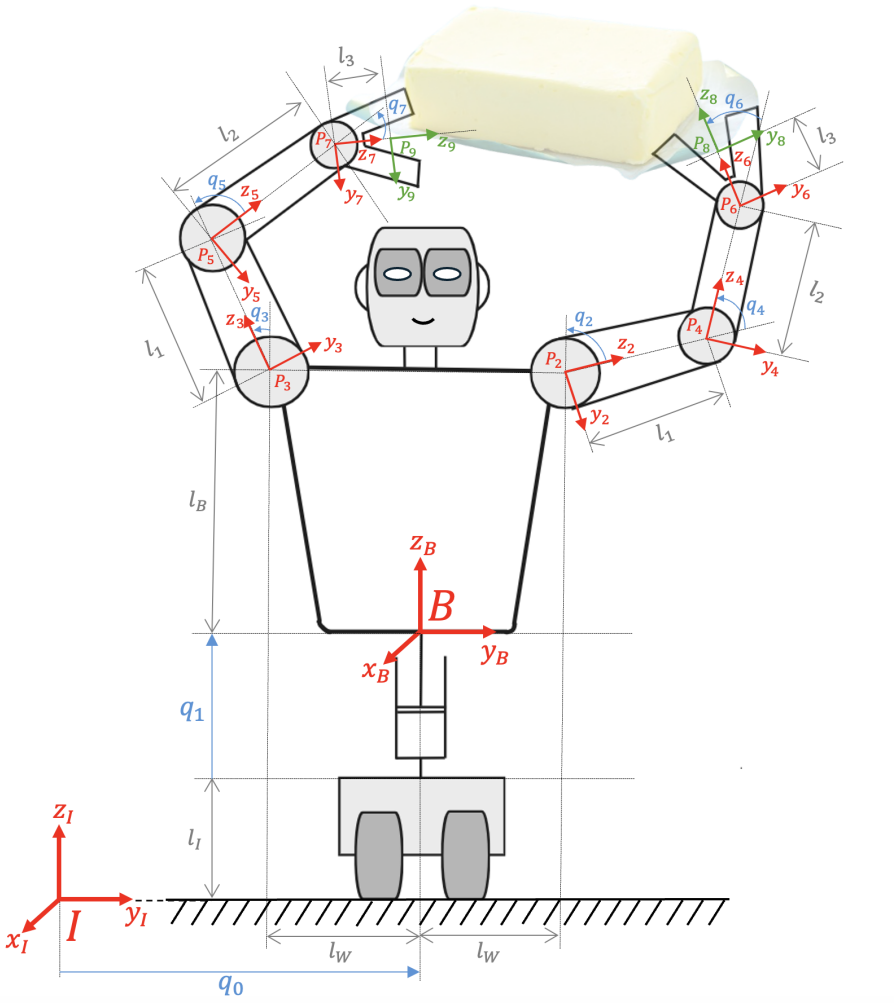


Figure 1: Schematic of a humanoid robot with two three degrees of freedom arms featuring unactuated grippers and a height-adjustable base. The base can move linearly along the y_I and z_I directions. All revolute joints on the arms rotate around the positive x_I axis. The x axis of frames $\{P_2\}, \{P_3\}, \ldots, \{P_7\}$ is parallel to the x_I axis.

2 Questions

The eccentric scientist, Dr. Richard, often faced a recurring problem — the butter was always just out of reach on the dining table. Determined to fix this, he designed a small humanoid robot capable of moving around and passing condiments. However, before he could develop its controller, he was called away on an intergalactic mission, leaving the task to you.

In this quiz, you will model the dynamics of the humanoid robot shown in Fig. 1. The robot has two arms with 3 Degrees-of-Freedom (DoF) each, connected to a mobile, height-adjustable base. Each arm has an unactuated gripper.

The base can move linearly along the y_I and z_I axes . The reference frame attached to the base is denoted as $\{B\}$. The arms are composed of three links each. The reference frames attached to each link are denoted as $\{P_2\}, \{P_3\}, \ldots, \{P_9\}$.

The generalized coordinates are defined as

$$\boldsymbol{q} = \begin{bmatrix} q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7 \end{bmatrix}^T \in \mathbb{R}^8 \ . \tag{1}$$

Clarification: The angles $q_2, q_3, \cdots, q_7$ are measured along the positive x-axis using right-hand thumb rule. For the state of the scene shown in Fig. 1, q_2, q_5 and q_7 would have negative values, and q_3, q_4 and q_6 would have positive values.

In the following questions, all required parameters are passed to your functions in a structure called `params`. You can access it as follows:

```
1 lI = params.lI;
2 lW = params.lW;
3 lB = params.lB;
4 l1 = params.l1;
5 l2 = params.l2;
6 l3 = params.l3;
```

Remember that MATLAB indexing starts from 1, so for example $q(1) = q_0$ in our robot description.

Question 1. 6 P.

Calculate the mass matrix $\boldsymbol{M}(\boldsymbol{q})$, nonlinear terms $\boldsymbol{b}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ (Coriolis and centrifugal), and gravitational terms $\boldsymbol{g}(\boldsymbol{q})$ of the full robot. A helper function, `dAdt()`, is available in the `utils` folder, to compute the time derivative of a matrix.

Implement your solution in `Q1_generate_eom.m`. Kinematics equations and dynamic parameters are provided and loaded at the beginning of the file.

Question 2. 4 P.

Implement a forward dynamics simulator for the **the whole system**.
To do this, we'll compute the arm joint accelerations $\ddot{\boldsymbol{q}}$ and integrates them to get $\dot{\boldsymbol{q}}$ and $\boldsymbol{q}$ at the next time step. You can use the the provided `M_fun_solution`, `b_fun_solution`, and `g_fun_solution` for your implementation.

Implement your solution in `Q2_forward_dynamics.m`. You can test your implementation using the provided simulator `Q2_simulator.p`. The simulator accepts a flag `use_solution` to switch between your implementation and the reference solution. To compare against the reference, call: `Q2_simulator(true)`. To test your own implementation, call: `Q2_simulator(false)`.

Question 3. 4 P.

In the previous question, you implemented the forward dynamics of Dr. Richard's robot. Unfortunately, without any control, the robot drifted aimlessly and occasionally struck a dramatic, unintended pose. To give it purpose, we will now implement a controller that allows it to intentionally perform one specific, glorious motion: the **dab**.



Figure 2: The dab.

Your goal is to design a joint-level PD controller with gravity compensation that makes the robot follow a desired reference trajectory — the dab pose. The controller should compute the control torques (or forces for prismatic joints) $\boldsymbol{\tau}$ that track the desired joint positions $\boldsymbol{q}^d$ and velocities $\dot{\boldsymbol{q}}^d$ based on the current states $\boldsymbol{q}$ and $\dot{\boldsymbol{q}}$. The gravitational effects can be obtained through the provided function `g_fun_solution(q)`. Use the proportional and derivative gains from the given `params` structure.

Implement your solution in `Q3_gravity_compensation.m`. You can test your implementation using the provided simulator `Q3_simulator.p`. The simulator accepts a flag `use_solution` to switch between your implementation and the reference solution. To compare against the reference, call: `Q3_simulator(true)`. To test your own implementation, call: `Q3_simulator(false)`.

Question 4. 4 P.

After mastering the dab, Dr. Richard turned his attention back to his breakfast problem. He decided to automate the ultimate act of culinary laziness — spreading his own brand of butter, proudly named **Marco Butter™**. Your mission: program the robot to gracefully hold a block of butter in its **left end-effector** and move it side-to-side, so the scientist never again needs to slide his knife back and forth.

However, during testing, an unfortunate mishap occurred: the robot's **right arm broke**. Therefore, your controller must **not command any torques** to the joints of the right arm. Make sure your implementation respects this hardware constraint by setting the corresponding torque entries to zero.

Additionally, an external force is applied to the butter along the z-axis of the inertial frame I (simulating the gentle resistance of spreading butter). Assume that this external force can be measured perfectly. Modify your controller so that it compensates for this disturbance in the computed control torques $\boldsymbol{\tau}$.

To achieve this automated butter-spreading, you will design a controller capable of accurately tracking a desired end-effector trajectory while compensating for the robot's full dynamics and for external forces acting on the butter held in the left gripper. Implement an **inverse dynamics controller** that produces the appropriate control torques $\boldsymbol{\tau}$ for each joint.

The inputs to this controller are the desired end-effector position, desired end-effector orientation, desired end-effector linear velocity, the required compensating force, as well as the current joint positions $\boldsymbol{q}$ and velocities $\dot{\boldsymbol{q}}$. The desired trajectory and compensating force have the following components (all expressed in the inertial frame):

- desired end-effector position ${}_I r^d_{IE} \in \mathbb{R}^3$
- desired end-effector velocity ${}_I v^d_E \in \mathbb{R}^3$
- desired end-effector orientation $C^d_{IE} \in \mathbb{R}^{3\times3}$
- desired compensating z-axis force ${}_I F^d_{E_z} \in \mathbb{R}$

Use the mass matrix, non-linear, and gravitational terms obtained from `M_fun_solution(q)`, `b_fun_solution(q, dq)`, and `g_fun_solution(q)`. The PD gains are provided in the parameters (`params`). You may also use the helper functions from the `utils` folder:

- `pseudoInverseMat()`, to compute a damped pseudo-inverse for any input matrix,
- `rotMatToRotVec()`, to compute the orientation error vector from a rotation matrix.

Implement your solution in `Q4_inverse_dynamics_control.m`. You can test your implementation using the provided simulator `Q4_simulator.p`. The simulator accepts a flag `use_solution` to switch between your implementation and the reference solution. To compare against the reference, call: `Q4_simulator(true)`. To test your own implementation, call: `Q4_simulator(false)`.

Hint: What mapping can be used to project forces in the task space into the joint space?